

SEMANA 3 ACADEMIA MONTERREY

- JUnit 5 Mockito Coverage

En el día 4 y 5 de la semana 3 de la academia se tocó el tema de pruebas unitarias en Java. Las pruebas unitarias se realizan utilizando el framework JUnit. El propósito de crear pruebas unitarias es para saber si nuestro código funciona de la manera esperada sin tener que realizar pruebas manuales en escenarios específicos. En otras palabras, se trata de pruebas automatizadas.

En nuestro código tenemos una abundancia de ejemplos sobre cómo se realizan las pruebas unitarias. Explicado de una manera simple, escribimos un código que se encargue de verificar que el resultado de una función o método nos de un resultado en concreto. Al realizar la prueba, podemos saber si nuestro código en verdad hace lo que esperamos o si existe un error en la redacción del mismo. Por ejemplo:

```
@BeforeEach
void prepararBoleta() {
    boleta = new Boleta(new Alumno("A01", "Ana
Torres"));
    boleta.registrar("Java", 90);
    boleta.registrar("SQL", 70);
}

@Test
@DisplayName("Igualdad, negacion y nulos")
void lasBasicas() {
    assertEquals(80.0, boleta.promedio()); //
(90 + 70) / 2
    assertEquals(2, boleta.totalMaterias());
```

```
assertTrue(boleta.aprobado());  
assertFalse(boleta.calificaciones().isEmpty());  
  
assertNotNull(boleta.alumno());  
  
assertNull(boleta.calificaciones().get("Kotlin"),  
           "Kotlin no se registro: el mapa devuelve  
null, no lanza excepcion");  
}
```

Al principio, nos encargamos de escribir una función con valores predeterminados. Después, en el apartado de test:

- La función `assertEquals()` se utiliza para verificar que una operación nos de un resultado específico, en este caso, un valor tipo `double` de 80.0 para el promedio de la boleta.
- Las funciones `assertTrue()` y `assertFalse()` se utilizan para verificar que una casilla en particular tenga un valor en particular o ninguno, lo cual nos indica que los resultados esperados se han registrado de manera correcta.
- `assertNotNull()` y `assertNull()` verifican que el registro tenga algún valor y no se quede en vacío, lo que nos puede ayudar a prevenir excepciones como `null pointer exception`.

Existe una variedad de aserciones que pueden utilizarse para realizar pruebas en JUnit. JUnit 5 es la versión del framework que utilizamos en nuestro programa, pero la versión más reciente es JUnit 6, liberada en 2025.

También se debe mencionar que una prueba no siempre espera un resultado aprobado. En ocasiones realizaremos pruebas esperando a que fallen con el propósito de comprobar que un método funciona o no de manera correcta. El que pase un test o no solo es una indicación de lo que estamos pidiendo resultó de una manera en particular, y depende de nosotros interpretar esos resultados como correctos o incorrectos dependiendo de lo que esperamos recuperar.

- **Mockito**

Antes de hablar sobre Mockito, debemos conocer el concepto de pruebas integradoras. A diferencia de las pruebas unitarias, que solamente se enfocan en probar el funcionamiento de una unidad de software específica, una prueba integradora busca conocer el funcionamiento de diferentes incorporaciones de software trabajando juntas, por ejemplo, verificar que una petición que se realice a una base de datos desde un tercero nos dé una respuesta del controlador encargado de recibir la petición y de que se obtenga un valor de la base de datos. Son pruebas que se realizan una vez que las pruebas unitarias se han verificado correctamente y están listas para seguir al próximo nivel.

Aquí es donde entra Mockito. Mockito es un framework de testing en Java que sirve para crear objetos de prueba para pruebas unitarias. Es decir, nosotros podemos crear objetos que simulan ciertos resultados de algún repositorio o base de datos y fingir que se obtuvo tal respuesta específica de ellos. Esto nos permite poder continuar con la prueba unitaria de nuestro sistema sin tener que verificar el funcionamiento de otro sistema diferente; nosotros especificamos mediante un mock el funcionamiento de un objeto para continuar con nuestras pruebas.

```
class CaptorTest {  
  
    private static final Alumno ANA = new  
Alumno("A01", "Ana Torres", "ana@academynty.mx");  
  
    @Mock private RepositorioAlumnos repoAlumnos;  
    @Mock private RepositorioCursos repoCursos;  
    @Mock private Notificador notificador;  
  
    @InjectMocks private ServicioInscripcion  
servicio;
```

```
/** @Captor evita escribir  
ArgumentCaptor.forClass(...) y ademas conserva los  
genericos. */
```

```
@Captor private ArgumentCaptor<Alumno>  
alumnoCapturado;  
  
@Captor private ArgumentCaptor<String>  
motivoCapturado;
```

```
@BeforeEach
```

```
void curso() {  
    java101 = new Curso("JAVA-101", 2);  
}
```

```
@Test
```

```
@DisplayName("A quien se le mando la confirmacion,  
exactamente")
```

```
void aQuienSeLeMando() {
```

```
when(repoAlumnos.buscar("A01")).thenReturn(Optional  
.of(ANA));
```

```
when(repoCursos.buscar("JAVA-101")).thenReturn(Optional  
onal.of(java101));
```

```
servicio.inscribir("A01", "JAVA-101");
```

```
// El captor se coloca DONDE iria el matcher,
dentro del verify:

verify(notificador).enviarConfirmacion(alumnoCapturado.capture(), any());

Alumno recibido = alumnoCapturado.getValue();
assertAll("el alumno que llego al notificador",
    () -> assertEquals("A01",
recibido.matricula()),
    () -> assertEquals("Ana Torres",
recibido.nombre()),
    () ->
assertTrue(recibido.correo().endsWith("@academynty.
mx"),
    "El correo tiene que ser del
dominio de la academia"));
}
```

En este ejemplo de nuestro código, buscamos probar la lógica del objeto servicioInscripción. Este servicio depende de otras tres clases: RepositorioAlumnos, Notificador, RepositorioCursos. Mockito nos ayuda a inyectar estas clases mediante objetos falsos marcados con @Mock. Al principio, dichos objetos están vacíos, pero los configuramos para que pretendan tener ciertos resultados. Por ejemplo, en la línea:

```
when(repoAlumnos.buscar("A01")).thenReturn(Optional.of(ANA));
```

Estamos pidiéndole a Mockito que cuando se utilice la función buscar en repoAlumnos, pretenda que encontró a Ana. Después, al correr el código de prueba, ServicioInscripción no sabe que estos objetos son mocks, simplemente los toma a partir de la interceptación de Mockito

para obtener un resultado esperado. De esta forma podemos confirmar que nuestro método aparece en nuestro log y de si pasó o no nuestra prueba.

En este ejemplo, además, no hay base de datos, no hay servidor de correo, no hay aplicación Spring Boot. Todo lo que es externo se reemplazó con mocks para verificar la lógica dentro de ServicioInscripcion, efectivamente convirtiéndola en una prueba unitaria.